molchemist

Molecule rendering from Molfile, SDF, and SMILES

v0.1.3 2026-08-03
MIT AND BSD-3-Clause AND Apache-2.0 AND (Apache-2.0 WITH LLVM-exception)

Render chemical structures from Molfile, SDF, and SMILES data.

Eito Yoneyama

`molchemist` renders chemical structures from Molfile / SDF data. It can also parse SMILES strings by generating a 2D layout first. The package turns molecular graphs into an `alchemist` drawing program and renders the final figure with CeTZ.

The package is aimed at Typst documents that need compact molecule figures, publication-oriented skeletal formulae, and light annotation without leaving Typst.

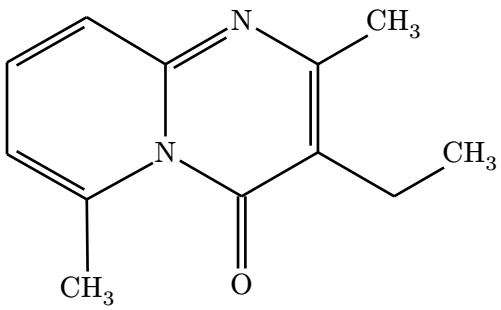
Table of Contents

Part I

Getting Started

Import `molchemist` and choose the renderer that matches your input: `#render-mol` for Molfile or SDF text, and `#render-smiles` for inline SMILES text.

```
#import "@preview/molchemist:0.1.3": *

#let mol-data = read("Structure2D_COMPOUND_CID_93406.sdf")
#render-mol(mol-data, abbreviate: true)
```



The examples below assume this import unless they need an additional package such as CeTZ.

On Typst 0.15.0 and later, `#render-mol` can also receive `path("molecule.sdf")` directly. This manual keeps using `read(...)` in examples for compatibility with older Typst versions.

Molfile/SDF records are detected as V2000 or V3000 automatically. When one SDF contains multiple structures, pass the one-based `record` option, for example `render-mol(sdf-data, record: 2)`. Missing records, empty structures, and malformed coordinate data report an error instead of producing an empty drawing.

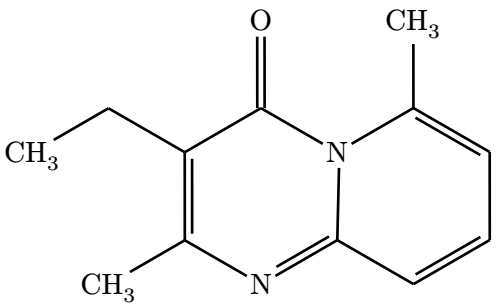
Usable 2D coordinates are preserved exactly. If bonded atoms collapse onto the same XY positions, a 3D record has no usable XY projection, or bond lengths are numerically unstable, `molchemist` generates a fresh 2D layout with Coordgen. Atom metadata, bond semantics, and stereochemical wedges/dashes still come from the selected SDF record.

SDF bond semantics are retained beyond the usual single, double, and triple orders. Aromatic and query bonds use dashed/dotted variants, any and `either` single bonds use a wavy line, undefined double-bond geometry uses a crossed double bond, coordination bonds retain their donor-to-acceptor arrow direction, and hydrogen bonds use a dotted line. SMILES quadruple bonds written with `$`, such as `[Cr]$[Cr]`, use four parallel lines. Long hydrogen bonds do not affect the covalent bond-length normalization used by the renderer. A wedge/dash bond to explicit hydrogen remains visible in abbreviated and skeletal modes. Atom `CFG` parity and V3000 enhanced stereo groups are retained as annotations below the structure and in dumped source.

SMILES is useful for compact inline examples or generated documents. Because SMILES stores connectivity rather than drawing coordinates, `molchemist` first computes a 2D layout and then renders the structure.

Dot-separated SMILES and disconnected Molfile/SDF graphs keep every component and place them side by side without inserting a visible operator. Atom and bond indices remain global across the complete input, so `⟨show-indices⟩` and annotation anchors work across component boundaries. Isolated hydrogen and carbon-only components also remain visible in abbreviated and skeletal modes.

```
#render-smiles(
  "CCC1=C(N=C2C=CC=C(N2C1=O)C)C",
  abbreviate: true,
)
```



I.1 Choosing an Input Format

- `Molfile / SDF`: coordinate-bearing structure text. Use it for database exports or drawing-tool output when preserving the supplied layout matters.
- `SMILES`: compact inline source text. Use it for examples, generated documents, and quick sketches. `molchemist` computes 2D coordinates before rendering.

Part II

Compatibility and Test Coverage

Surface	Minimum	Continuously tested
Typst package with `str` or `bytes` input	`0.14.0`	`0.14.0`, `0.14.1`, `0.14.2`, `0.15.0`, and `0.15.1`
Typst `path(...)` input	`0.15.0`	`0.15.0` and `0.15.1`
`molchemist-cli` build	Rust `1.86`	Rust `1.86` on Ubuntu, macOS, and Windows

For each listed Typst version, CI compiles the rendering fixtures on Ubuntu and checks that package dump output is byte-for-byte identical to CLI output. Rust parser, formatter, CLI, and WASM-host tests additionally run on macOS and Windows. The matrix covers the published package entrypoint and CLI-generated source, not the separate third-party toolchain used to typeset this manual.

> Compatibility with Typst 0.14.0 and 0.14.1 is tested for users who cannot upgrade, but those releases contain an upstream WebAssembly runtime security issue[1]. Prefer Typst 0.14.2 or later for production documents.

[1] https://github.com/typst/typst/releases/tag/v0.14.2

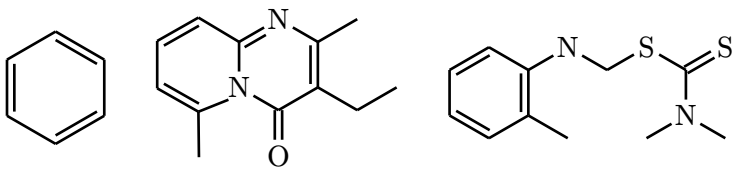
Part III

Example Data

The SDF examples in this manual use real PubChem records included in the repository test data.

- PubChem CID 241[2]: benzene.
- PubChem CID 93406[3]: `3-ethyl-2,6-dimethylpyrido[1,2-a]pyrimidin-4-one`.
- PubChem SID 93298[4]: a deposited DTP/NCI substance record associated with CID 235403.

```
#let benzene = read("Structure2D_COMPOUND_CID_241.sdf")
#let fused = read("Structure2D_COMPOUND_CID_93406.sdf")
#let deposited = read("DepositedStructure_SUBSTANCE_SID_93298_Version_3.sdf")

#grid(
  columns: 3,
  gutter: 7mm,
  align: center + horizon,
  render-mol(benzene, skeletal: true, config: (atom-sep: 1.55em)),
  render-mol(fused, skeletal: true, config: (atom-sep: 1.55em)),
  render-mol(deposited, skeletal: true, config: (atom-sep: 1.55em)),
)
```



[2] https://pubchem.ncbi.nlm.nih.gov/compound/241
[3] https://pubchem.ncbi.nlm.nih.gov/compound/93406
[4] https://pubchem.ncbi.nlm.nih.gov/substance/93298

Part IV

Input and Semantic Fidelity

The examples in this chapter focus on information that is easy to lose when a molecular file is reduced to a generic graph. Each example shows both the Typst source and the resulting drawing so that input selection, metadata, bond meaning, and stereochemistry can be checked independently.

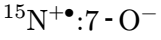
IV.1 SDF Versions and Record Selection

`#render-mol` detects V2000 and V3000 independently for every selected SDF record. The `⟨record⟩` option is one-based and defaults to `1`; it does not depend on the format of preceding records. This makes mixed-version SDF collections usable without preprocessing.

```
#let records = read("structures.sdf")

#grid(
  columns: 2,
  gutter: 10mm,
  align: center + top,
  [
    *V2000 · record 1*
    #v(2mm)
    #render-mol(records, record: 1)
  ],
  [
    *V3000 · record 2*
    #v(2mm)
    #render-mol(records, record: 2, abbreviate: true)
  ],
)
```

V2000 · record 1

V3000 · record 2



The second synthetic record also demonstrates V3000 charge, isotope, radical, and atom-map fields. Bond configuration is covered separately in the stereochemistry examples below. An out-of-range record number, an empty structure, malformed CTAB data, or non-finite coordinates raises an explicit error instead of silently drawing the wrong record.

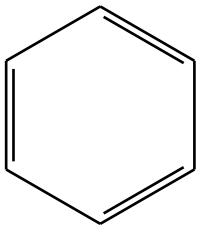
IV.2 Coordinate Preservation and Layout Recovery

Usable source coordinates are preserved, including their relative orientation. Automatic layout is requested only when the XY geometry is unusable—for example, when every bonded atom

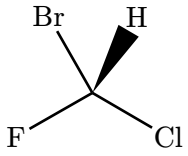
is collapsed onto one point or a nominally 3D record has no meaningful XY projection. The recovered drawing still uses the source bond orders, wedge direction, atom metadata, and record ordering.

```
#let supplied = read("benzene-2d.sdf")
#let collapsed = read("collapsed-layout.sdf")

#grid(
  columns: 2,
  gutter: 12mm,
  align: center + top,
  [
    *Preserved 2D coordinates*
    #v(2mm)
    #render-mol(supplied, skeletal: true)
  ],
  [
    *Recovered collapsed coordinates*
    #v(2mm)
    #render-mol(collapsed, skeletal: true)
  ],
)
```

Preserved 2D coordinates



Recovered collapsed coordinates



The fallback is deliberately conservative: crowded but numerically valid coordinates are not redrawn merely because their full-mode labels overlap. Change `⟨atom-sep⟩` or use abbreviated/skeletal mode when the geometry is valid but the typography is dense.

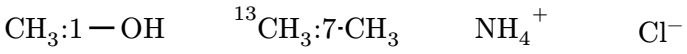
IV.3 Atom Metadata and Explicit Labels

Bracket-atom metadata remains structured through the SMILES parser, WASM boundary, formatter, and Typst renderer. Isotope numbers and formal charges use their conventional upper positions, hydrogen counts use a lower position, and atom classes remain visible as an inline `:n` suffix. Invisible balancing attachments keep the primary atom symbols aligned when differently scripted labels appear in one disconnected structure.

```
#grid(
  columns: 3,
```

```
  gutter: 8mm,
  align: center + top,
  [
    *Atom class*
    #v(2mm)
    #render-smiles("[CH3:1]O", abbreviate: true)
  ],
  [
    *Isotope + class*
    #v(2mm)
    #render-smiles("[13CH3:7]C", abbreviate: true)
  ],
  [
    *Charge + H count*
    #v(2mm)
    #render-smiles("[NH4+].[Cl-]", skeletal: true)
  ],
)
```

Atom class **Isotope + class** **Charge + H count**



In full mode, explicitly represented hydrogen atoms remain separate graph nodes. In abbreviated and skeletal modes, foldable hydrogens join their parent labels, except when folding would erase a stereochemical wedge/dash or change the meaning of a bridging hydrogen.

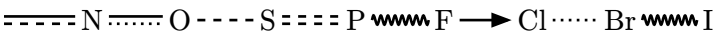
IV.4 Bond Semantics

V3000 bond orders `4` through `10` remain distinct: aromatic, single-or-double, single-or-aromatic, double-or-aromatic, any, coordination, and hydrogen bonds are not collapsed to ordinary single bonds. Query and aromatic bonds use dashed/dotted partial lines, any/either bonds use a wavy line, coordination bonds preserve their donor-to-acceptor direction with a filled arrowhead, and hydrogen bonds use a dotted line. A SMILES `$` bond is rendered separately as four parallel lines.

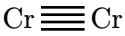
V3000 order	Meaning	Visual cue
`4`	Aromatic	Double line with a dashed partial line
`5`	Single or double	Double line with a dotted partial line
`6`	Single or aromatic	Dashed single line
`7`	Double or aromatic	Dashed double line
`8`	Any	Wavy line
`9`	Coordination	Directed line with a filled arrowhead
`10`	Hydrogen	Dotted line

```
#let extended = read("bond-semantics.sdf")

#grid(
  columns: 1,
  row-gutter: 5mm,
  align: center,
  [
    *V3000 extended bond orders*
    #v(2mm)
    #render-mol(
      extended,
      abbreviate: true,
      config: (atom-sep: 3.0em),
    )
  ],
  [
    *OpenSMILES quadruple bond*
    #v(2mm)
    #render-smiles("[Cr]$[Cr]")
  ],
)
```

V3000 extended bond orders



OpenSMILES quadruple bond

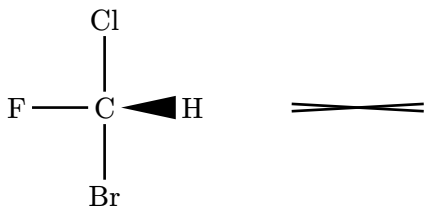


Long hydrogen bonds are excluded when the renderer computes a representative covalent bond length. This prevents a distant noncovalent contact from shrinking the covalent part of the structure.

IV.5 SDF Stereochemical Metadata

For Molfile/SDF input, up/down single bonds remain wedge/dash bonds. Undefined double-bond geometry—V2000 stereo code `3` or V3000 double-bond `CFG=2`—uses a crossed double bond. Atom `CFG` parity and V3000 `STEABS`, `STEREL`, and `STERAC` collections are retained as annotations rather than being discarded.

```
#let stereo-data = read("stereochemistry.sdf")
#render-mol(stereo-data, skeletal: true)
```



Stereo annotations: C CFG=1; OR7 (a0)

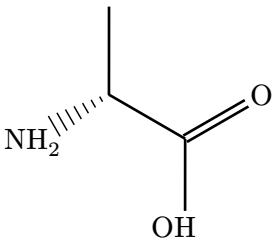
The explicit hydrogen above is intentionally kept visible because its wedge carries stereochemical information. The disconnected crossed double bond in the same record also demonstrates that component separation does not drop bond configuration.

IV.6 SMILES Tetrahedral and Double-Bond Stereo

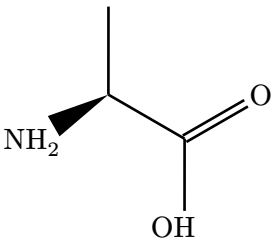
OpenSMILES `@` and `@@` are interpreted using local SMILES neighbor order, not by assigning a fixed wedge to one token. Branch order, bracket hydrogens, incoming atoms, and ring-closure token positions therefore contribute to the depicted configuration. Directional `/` and `\` bonds are likewise resolved as a pair around the double bond.

```
#let stereo-card(title, source) = align(center)[
  *#title*
  #v(2mm)
  #render-smiles(source, skeletal: true)
]

#grid(
  columns: 2,
  gutter: 10mm,
  row-gutter: 6mm,
  align: top,
  stereo-card([D-alanine · (R)], "N[C@H](C)C(=O)O"),
  stereo-card([L-alanine · (S)], "N[C@@H](C)C(=O)O"),
  stereo-card([E-difluoroethene], "F/C=C/F"),
  stereo-card([Z-difluoroethene], "F/C=C\\F"),
)
```

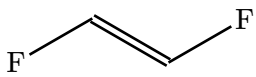
D-alanine · (R)



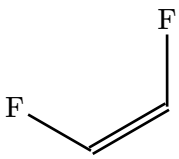
L-alanine · (S)



E-difluoroethene



Z-difluoroethene



The implicit-hydrogen form `N[C@@H](C)C(=O)O` and the explicit-hydrogen form `N[C@@]([H])(C)C(=O)O` describe the same local configuration and are tested to produce the same absolute orientation.

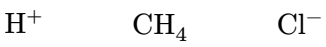
IV.7 Multi-component Structures

Disconnected Molfile/SDF graphs and dot-separated SMILES retain every component in source order. Components are separated by whitespace with no visible synthetic operator. Atom and bond indices remain global, so annotation anchors can target later components without renumbering them.

```
#grid(
  columns: 1,
  row-gutter: 5mm,
  align: center,
  [
    *Dot-separated salt*
    #v(2mm)
    #render-smiles("[Na+].[Cl-]", abbreviate: true)
  ],
  [
    *Visible isolated components*
    #v(2mm)
    #render-smiles("[H+].C.[Cl-]", skeletal: true)
  ],
)
```

Dot-separated salt

Visible isolated components



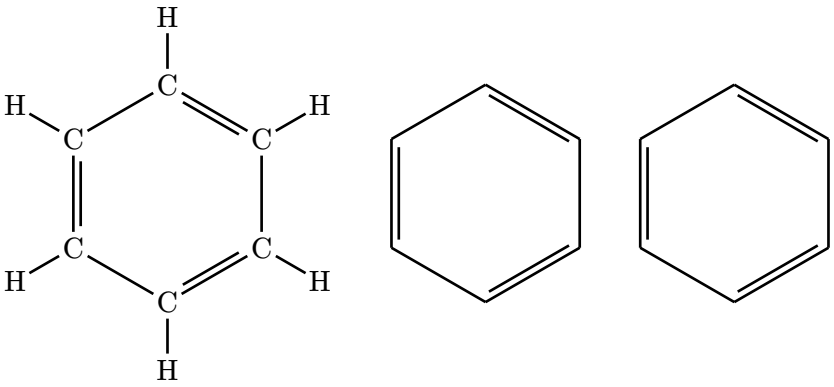
Isolated hydrogen and zero-heavy-neighbor carbon components remain visible in abbreviated and skeletal modes. Scripted labels are balanced around their primary symbols, so `H`, `CH`, and `Cl` align while the `4` in `CH4` remains correctly lowered.

Part V

Rendering Modes

Both renderers support the same three modes. Full mode is useful for small molecules and debugging. Abbreviated mode folds common hydrogens and terminal carbons into labels. Skeletal mode is usually the best default for publication figures.

```
#let mol-data = read("Structure2D_COMPOUND_CID_241.sdf")

#grid(
  columns: 3,
  gutter: 8mm,
  align: center + horizon,
  render-mol(mol-data),
  render-mol(mol-data, abbreviate: true),
  render-mol(mol-data, skeletal: true),
)
```

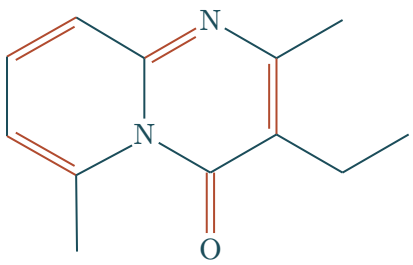


Part VI

Styling

Pass `⟨config⟩` for visual adjustments that should reach `alchemist`, such as atom spacing, fragment color, and bond styles.

```
#let mol-data = read("Structure2D_COMPOUND_CID_93406.sdf")

#render-mol(
  mol-data,
  skeletal: true,
  config: (
    atom-sep: 2.6em,
    fragment-color: rgb("#1b4d5a"),
    single: (stroke: 0.75pt + rgb("#1b4d5a")),
    double: (stroke: 0.75pt + rgb("#b14b2f")),
  ),
)
```



Molfile and SDF input normally contains usable coordinates; collapsed or numerically unstable coordinates receive an automatic 2D fallback layout. Routing-oriented `alchemist` settings do not reshape either graph; prefer `atom-sep`, font, and stroke settings for visual tuning.

Part VII

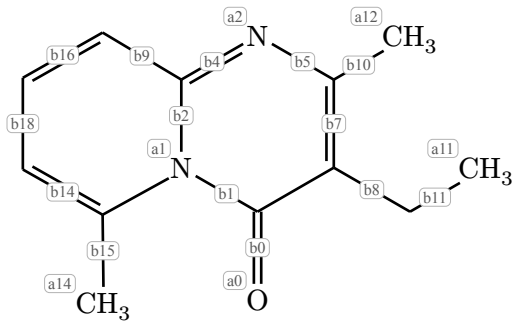
Annotations

Annotations are drawn after the molecule. They are meant for sparse publication callouts, not for replacing a full figure editor.

VII.1 Finding Atom and Bond Indices

Enable `⟨show-indices⟩` while authoring. The visible labels are the indices used by `#atom-anchor` and `#bond-anchor`.

```
#let mol-data = read("Structure2D_COMPOUND_CID_93406.sdf")
#render-mol(mol-data, abbreviate: true, show-indices: true)
```

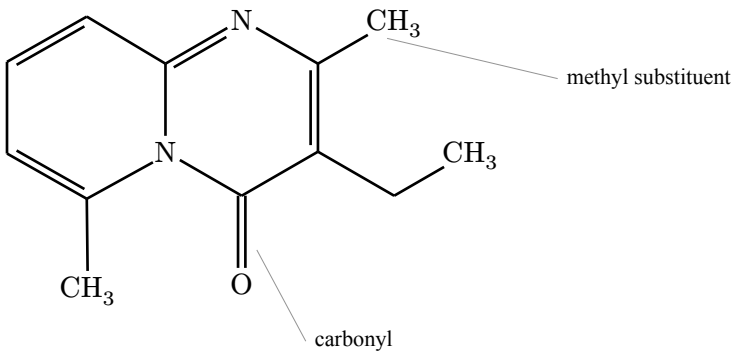


VII.2 Callouts

Use `#callout-annotation` for external labels. Defaults are intentionally quiet: no arrowhead, a thin leader line, and an unboxed label.

```
#let mol-data = read("Structure2D_COMPOUND_CID_93406.sdf")

#render-mol(
  mol-data,
  abbreviate: true,
  annotations: (
    callout-annotation(
      bond-anchor(0),
      [carbonyl],
      label-at: (to: molecule-anchor(anchor: "south"), rel: (0.82, -0.48)),
      leader-start: (to: molecule-anchor(anchor: "south"), rel: (0.7, -0.48)),
      leader-end: (to: bond-anchor(0), rel: (0.2, -0.24)),
      leader: "curve",
      stroke: luma(58%) + 0.28pt,
      label-size: 0.76em,
      label-anchor: "west",
```

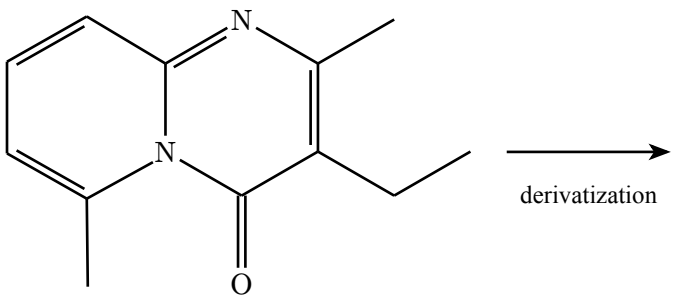
```
    ),
    callout-annotation(
      atom-anchor(12),
      [methyl substituent],
      label-at: (to: molecule-anchor(anchor: "east"), rel: (0.9, 1.04)),
      leader-start: (to: molecule-anchor(anchor: "east"), rel: (0.78, 1.04)),
      leader-end: (to: atom-anchor(12), rel: (0.28, -0.38)),
      leader: "curve",
      stroke: luma(58%) + 0.28pt,
      label-size: 0.76em,
      label-anchor: "west",
    ),
  ),
)
```



VII.3 Arrows

Use `#arrow-annotation` for molecule-level process arrows or simple directional marks.

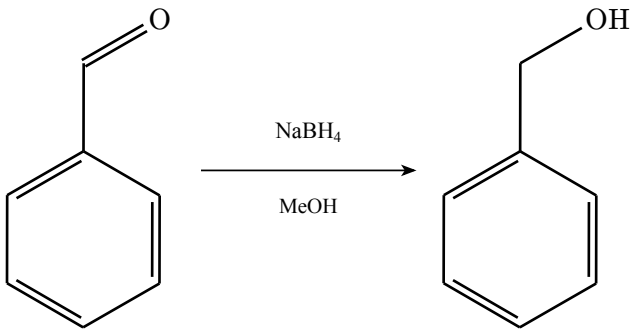
```
#let mol-data = read("Structure2D_COMPOUND_CID_93406.sdf")

#render-mol(
  mol-data,
  skeletal: true,
  annotations: arrow-annotation(
    (to: molecule-anchor(anchor: "east"), rel: (0.48, 0)),
    (to: molecule-anchor(anchor: "east"), rel: (2.65, 0)),
    label: [derivatization],
    label-offset: (0, -0.46),
    label-anchor: "north",
  ),
)
```



VII.4 Reaction Schemes

For reactions, compose separate molecule renderings with normal Typst/CeTZ layout. This keeps reaction arrows independent from molecule annotations.

```
#let reaction-arrow(above, below: none) = cetz.canvas({
  import cetz.draw: *
  line(
    (0.1, 0),
    (2.95, 0),
    stroke: 0.65pt + black,
    mark: (end: ">>", scale: 0.72, fill: black),
  )
  content((1.52, 0.42), text(size: 0.78em)[#above], anchor: "south")
  if below != none {
    content((1.52, -0.38), text(size: 0.72em)[#below], anchor: "north")
  }
})

#grid(
  columns: (auto, 28mm, auto),
  column-gutter: 4mm,
  align: horizon + center,
  render-smiles("O=CC1=CC=CC=C1", abbreviate: true),
  reaction-arrow([NaBH#sub[4]], below: [MeOH]),
  render-smiles("OCC1=CC=CC=C1", abbreviate: true),
)
```



VII.5 Mechanism Example: von Richter Reaction

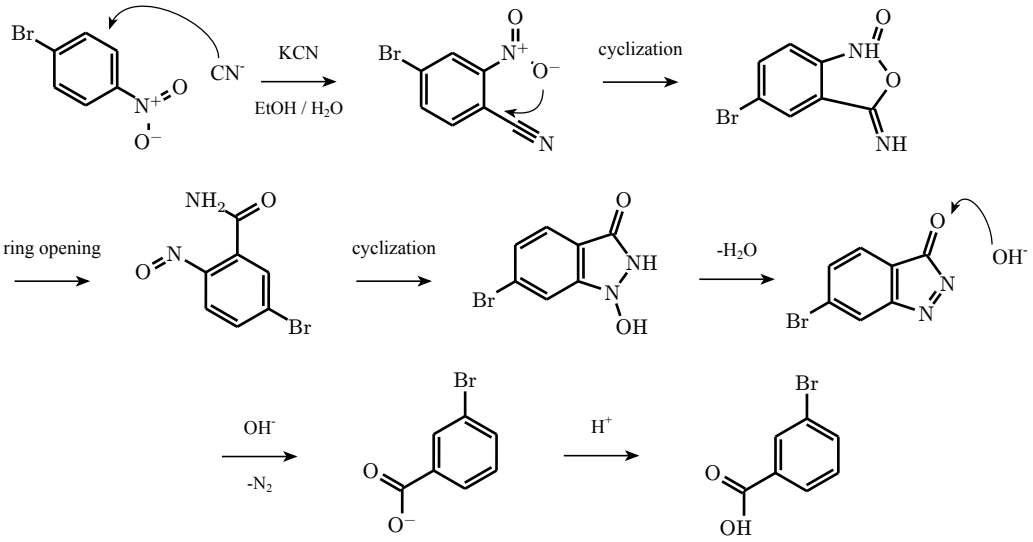
Long mechanisms can combine SMILES rendering, `#cetz-annotation` for electron movement, and ordinary CeTZ layout. This example shows the classical conversion of p-bromonitrobenzene to m-bromobenzoic acid.

```
// Electron-flow helpers.
#let electron-arrows(body) = cetz-annotation(mol => {
  import cetz.draw: *
  body(mol)
})

#let electron-arrow(
  mol,
  from,
  to,
  from-offset: (0, 0),
  to-offset: (0, 0),
  controls: ((0.35, 0.6), (0.35, 0.6)),
) = {
  import cetz.draw: *
  let start = (to: (name: mol, anchor: from), rel: from-offset)
  let end = (to: (name: mol, anchor: to), rel: to-offset)
  if controls.len() == 1 {
    bezier(
      start,
      end,
      (to: start, rel: controls.at(0)),
      stroke: 0.48pt + black,
      mark: (end: ">>", scale: 0.62, fill: black),
    )
  } else {
    bezier(
      start,
      end,
      (to: start, rel: controls.at(0)),
      (to: end, rel: controls.at(1)),
      stroke: 0.48pt + black,
      mark: (end: ">>", scale: 0.62, fill: black),
    )
  }
}

#let reagent(mol, at, label, offset: (0, 0)) = {
  import cetz.draw: *
  content(
    (to: (name: mol, anchor: at), rel: offset),
    text(size: 8pt)[#label],
    anchor: "center",
  )
```

```
}

// Reaction-arrow and row-layout helpers.
#let reaction-arrow(above: none, below: none) = cetz.canvas({
  import cetz.draw: *
  line(
    (0.08, 0),
    (1.08, 0),
    stroke: 0.6pt + black,
    mark: (end: ">>", scale: 0.68, fill: black),
  )
  if above != none {
    content((0.58, 0.3), text(size: 7.2pt)[#above], anchor: "south")
  }
  if below != none {
    content((0.58, -0.27), text(size: 6.8pt)[#below], anchor: "north")
  }
})

#let molecule(smiles, annotations: none) = box(
  width: 32mm,
  align(center)[
    #set text(size: 8pt)
    #render-smiles(
      smiles,
      abbreviate: true,
      config: (atom-sep: 1.7em),
      annotations: annotations,
    )
  ],
)

#let mechanism-row(..items) = {
  let cells = items.pos()
  grid(
    columns: cells.map(
      item => if item.at(0) == "molecule" { 32mm } else { 12mm },
    ),
    column-gutter: 0.6mm,
    align: horizon + center,
    ..cells.map(item => item.at(1)),
  )
}

// Substrate and early intermediates.
#let substrate = molecule(
  "O=[N+]([O-])c1ccc(Br)cc1",
  annotations: electron-arrows(mol => {
    import cetz.draw: *
    reagent(mol, "east", [CN#super[-]], offset: (0.52, 0.22))
```

```
    electron-arrow(
      mol,
      "east",
      "b4.50%",
      from-offset: (0.4, 0.42),
      to-offset: (0.1, 0.16),
      controls: ((0, 0.48), (0.18, 0.68)),
    )
  }),
)

#let sigma-adduct = molecule(
  "O=[N+]([O-])C1=CC(Br)=CC=C1C#N",
  annotations: electron-arrows(mol => {
    electron-arrow(
      mol,
      "a2.south",
      "b10.20%",
      from-offset: (-0.05, -0.08),
      to-offset: (0.18, 0),
      controls: ((0, -0.4),),
    )
  }),
)

#let cyclic-imidate = molecule("N=C1ON(=O)c2ccc(Br)cc21")

// Later intermediates and products.
#let nitroso-amide = molecule("O=Nc1c(C(=O)N)cc(Br)cc1")
#let hydroxy-azo = molecule("O=C1NN(O)C2=C1C=CC(Br)=C2")

#let azoketone = molecule(
  "O=C1N=NC2=C1C=CC(Br)=C2",
  annotations: electron-arrows(mol => {
    import cetz.draw: *
    reagent(mol, "east", [OH#super[-]], offset: (0.72, 0.18))
    electron-arrow(
      mol,
      "east",
      "b0.50%",
      from-offset: (0.48, 0.34),
      to-offset: (0.28, 0.38),
      controls: ((0, 0.36), (0.16, 0.5)),
    )
  }),
)

#let carboxylate = molecule("O=C([O-])c1cc(Br)ccc1")
#let product = molecule("O=C(O)c1cc(Br)ccc1")
```

```
// Compose the mechanism.
#block(breakable: false, width: 100%)[
  #stack(
    dir: ttb,
    spacing: 5mm,
    mechanism-row(
      ("molecule", substrate),
      ("arrow", reaction-arrow(above: [KCN], below: [EtOH / H#sub[2]O])),
      ("molecule", sigma-adduct),
      ("arrow", move(dy: -2.1mm, reaction-arrow(above: [cyclization]))),
      ("molecule", cyclic-imidate),
    ),
    mechanism-row(
      ("arrow", reaction-arrow(above: [ring opening])),
      ("molecule", nitroso-amide),
      ("arrow", reaction-arrow(above: [cyclization])),
      ("molecule", hydroxy-azo),
      ("arrow", reaction-arrow(above: [-H#sub[2]O])),
      ("molecule", azoketone),
    ),
    align(center)[
      #mechanism-row(
        ("arrow", reaction-arrow(above: [OH#super[-]], below: [-N#sub[2]])),
        ("molecule", carboxylate),
        ("arrow", move(dy: -2mm, reaction-arrow(above: [H#super[+]]))),
        ("molecule", product),
      )
    ],
  )
]
```

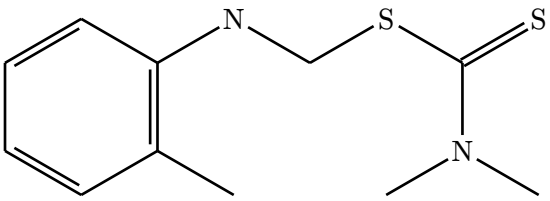


The mechanistic sequence follows M. Rosenblum, *The Mechanism of the von Richter Reaction*, J. Am. Chem. Soc. 82 (1960), 3796-3798, doi:10.1021/ja01499a090[5]. Molecule-specific anchors and curved-arrow routing can be adjusted directly in the figure source.

VII.6 Low-Level Labels

Use `#label-annotation` when no leader line is needed.

```
#let mol-data = read("DepositedStructure_SUBSTANCE_SID_93298_Version_3.sdf")

#render-mol(
  mol-data,
  skeletal: true,
  annotations: label-annotation(
    molecule-anchor(anchor: "south"),
    [PubChem SID 93298],
    offset: (0, -0.65),
    label-anchor: "north",
  ),
)
```



PubChem SID 93298

VII.7 Custom CeTZ Overlays

For final figure polishing, `#cetz-annotation` exposes the generated molecule name, so you can draw directly against CeTZ anchors.

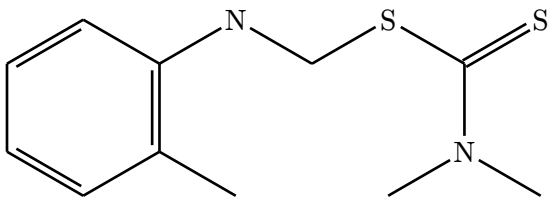
```
#let mol-data = read("DepositedStructure_SUBSTANCE_SID_93298_Version_3.sdf")

#render-mol(
  mol-data,
  skeletal: true,
  annotations: cetz-annotation(mol => {
    import cetz.draw: *
    content(
      (to: (name: mol, anchor: "north"), rel: (0, 0.54)),
      text(size: 0.82em)[database structure],
      anchor: "south",
```

[5] https://doi.org/10.1021/ja01499a090

```
    )
  }),
)
```

---

database structure



Part VIII

Dump Mode

With `⟨dump⟩`, `molchemist` returns generated `alchemist` source instead of a drawing. Use this when a figure needs manual surgery beyond the annotation API.

```
#let mol-data = read("Structure2D_COMPOUND_CID_241.sdf")
#render-mol(mol-data, skeletal: true, dump: true)
```

```
#let base-sep = 3em
#skeletize({
  hook("a0")
  branch({
    double(absolute: -29.997863113888922deg, atom-sep: base-sep *
    1.2346644251497605, offset: "right", name: "b0")
    hook("a1")
    single(absolute: -90deg, atom-sep: base-sep * 1.2345846673836163,
    name: "b3")
    hook("a3")
    double(absolute: -150.0021368861111deg, atom-sep: base-sep *
    1.2346644251497605, offset: "right", name: "b7")
    hook("a5")
    single(absolute: 149.99927221917264deg, atom-sep: base-sep *
    1.2345575062221579, name: "b9")
    hook("a4")
    double(absolute: 90deg, atom-sep: base-sep * 1.2345846673836163,
    offset: "right", name: "b5")
    hook("a2")
    branch({
      single(absolute: 0deg, atom-sep: 0pt, stroke: none, name: "a2-
      links", links: (
      "a0": single(absolute: 30.000727780827372deg, atom-sep: base-sep *
      1.2345575062221579, name: "b1"),
    ))
    })
  })
})
```

Part IX

Command-Line Export

For scripts and editor workflows, install the `molchemist-cli` crate with `cargo install --locked molchemist-cli`. Its `molchemist dump` command accepts Molfile, SDF, or SMILES input and writes the same formatted source as `⟨dump⟩` to standard output. Add `--standalone` to include the current `alchemist` import and an auto-sized page, or `--output figure.typ` to write directly to a file.

The normal output is an editable Alchemist program rather than an image or diagnostic transcript. For example, the command and its complete standard output are:

```
$ molchemist dump --smiles 'CC(=O)O' --mode skeletal
#let base-sep = 3em
#skeletize({
  hook("a0")
  single(absolute: 29.79036703670196deg, atom-sep: base-sep * 1, name:
  "b0")
  hook("a1")
  branch({
    double(absolute: 89.79373607661383deg, atom-sep: base-sep *
    1.000062954206203, name: "b1")
    fragment("O", name: "a2")
  })
  single(absolute: -30.20116835518715deg, atom-sep: base-sep *
  0.9999817671902098, name: "b2")
  fragment("OH", name: "a3")
})
```

Redirect this snippet when it will be imported or edited inside an existing document. Use `⟨record⟩` for a multi-record SDF, or request a complete compilable document with `⟨standalone⟩`:

```
$ molchemist dump molecule.sdf > molecule.typ
$ molchemist dump structures.sdf --record 2 --mode skeletal > record-2.typ
$ molchemist dump --smiles 'CC(=O)O' --mode skeletal \
    --standalone --output acetic-acid.typ
$ typst compile acetic-acid.typ
```

Input may also be piped from another program—for example, `printf '%s\n' 'c1ccccc1' | molchemist dump --format smiles > benzene.typ`. Diagnostics are written to standard error and generated source exclusively to standard output, so redirection does not mix warnings or errors into a `.typ` file.

Part X

Publication Guidance

For paper figures, start with `⟨skeletal⟩` for hydrocarbon-heavy structures and `⟨abbreviate⟩` when heteroatom hydrogens or terminal groups should remain explicit. Keep full mode for small structures and debugging.

Keep annotations sparse. In most cases, a thin unboxed `#callout-annotation` is better than a boxed label or an arrowhead. If a leader line looks like a chemical bond, move the label with `⟨label-at⟩` or stop the leader outside the structure with `⟨leader-end⟩`.

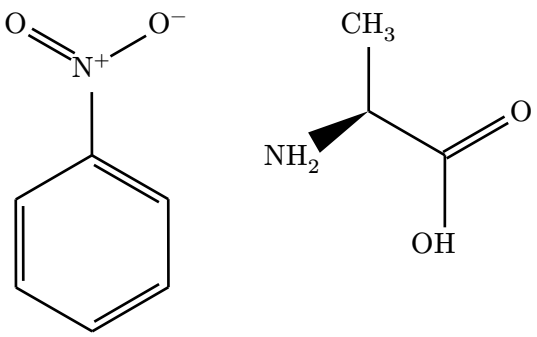
> Dense structures can still overlap in full mode, especially after SMILES implicit hydrogens are expanded. Prefer abbreviated or skeletal mode when a molecule is meant for a publication figure.

Part XI

SMILES Notes

SMILES support is a parse-and-layout pipeline. The parser accepts common SMILES notation, aromatic rings, charges, tetrahedral `@` / `@@` centers, and `/` / `\` double-bond geometry. It rejects malformed branch, dot, bond, bracket-property, charge, isotope, atom-class, directional-bond, and aromatic notation instead of normalizing it silently. Atom classes from `0` through `9999` are accepted, and aromatic systems must satisfy Hückel's rule and admit a valence-compatible Kekulé assignment. Tetrahedral centers retain OpenSMILES local neighbor order, including bracket hydrogens and ring-closure token positions.

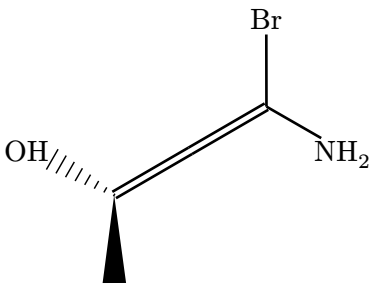
```
#grid(
  columns: 2,
  gutter: 10mm,
  align: top + center,
  render-smiles("O=[N+]([O-])c1ccccc1", abbreviate: true),
  render-smiles("N[C@@H](C)C(=O)O", abbreviate: true),
)
```



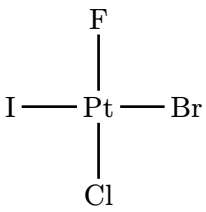
Extended OpenSMILES chirality classes are rendered geometrically when their topology permits an unambiguous projection. `@AL` uses terminal wedge/dash bonds, `@SP` uses its U/4/Z ligand path, and `@TB` / `@OH` combine the specified ligand winding with solid and hashed viewing-axis bonds. Invalid or cyclic topologies that cannot be rearranged safely retain their original chirality tag as a fallback annotation.

```
#let chiral-card(title, source) = align(center)[
  *#title*
  #v(2mm)
  #render-smiles(source, skeletal: true)
]

#grid(
  columns: 2,
  gutter: 8mm,
  row-gutter: 7mm,
  align: top,
  chiral-card([Allene · `@AL1`], "NC(Br)=[C@AL1]=C(O)C"),
  chiral-card([Square planar · `@SP2`], "[Pt@SP2](F)(Cl)(Br)I"),
  chiral-card([Trigonal bipyramidal · `@TB5`], "[As@TB5](F)(Cl)(Br)(N)S"),
  chiral-card([Octahedral · `@OH5`], "[Co@OH5](F)(Cl)(Br)(I)(N)S"),
)
```

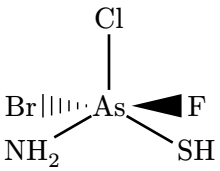
Allene · `@AL1`



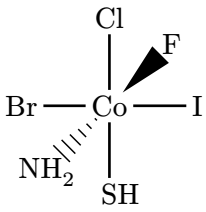
Square planar · `@SP2`



Trigonal bipyramidal · `@TB5`



Octahedral · `@OH5`



Part XII

API Reference

XII.1 Rendering Functions

```
#render-mol(
  (data),
  (abbreviate): false,
  (skeletal): false,
  (dump): false,
  (config): (:),
  (annotations): none,
  (show-indices): false
) → content
```

Render a molecule from raw Molfile or SDF text.

Usable input coordinates are preserved. Collapsed or numerically unstable coordinates receive a generated 2D layout while retaining source metadata and bond stereochemistry.

Argument

`(data)` — `str` | `bytes` | `path`

Raw `.mol` or `.sdf` data. Typst 0.15.0 and later may pass `path(...)` directly; older versions should pass `read(...)` output.

Argument

`(abbreviate): false` — `bool`

Enables abbreviated rendering.

Argument

`(skeletal): false` — `bool`

Enables skeletal rendering. This overrides `(abbreviate)`.

Argument

`(dump): false` — `bool`

Returns generated `alchemist` source code instead of rendering the molecule.

Argument

`(config): (:)` — `dictionary`

Visual configuration passed to `alchemist`.

Argument

`(annotations): none` — `annotation` | `array` | `none`

Optional overlay annotation or array of annotations.

Argument
`⟨show-indices⟩: false` `bool` | `str`

Debug overlay for annotation authoring. Use `true`, `"all"`, `"atoms"`, or `"bonds"`.

```
#render-smiles(
  ⟨smiles⟩,
  ⟨abbreviate⟩: false,
  ⟨skeletal⟩: false,
  ⟨dump⟩: false,
  ⟨config⟩: (:),
  ⟨annotations⟩: none,
  ⟨show-indices⟩: false
) → content
```

Parse a SMILES string, generate 2D coordinates, and render the resulting molecule.

Argument
`⟨smiles⟩` `str`

A SMILES string.

Argument
`⟨abbreviate⟩: false` `bool`

Enables abbreviated rendering after layout generation.

Argument
`⟨skeletal⟩: false` `bool`

Enables skeletal rendering. This overrides `⟨abbreviate⟩`.

Argument
`⟨dump⟩: false` `bool`

Returns generated `alchemist` source code instead of rendering the molecule.

Argument
`⟨config⟩: (:)` `dictionary`

Visual configuration passed to `alchemist`.

Argument
`⟨annotations⟩: none` `annotation` | `array` | `none`

Optional overlay annotation or array of annotations.

Argument
`⟨show-indices⟩: false` `bool` | `str`

Debug overlay for annotation authoring. Use `true`, `"all"`, `"atoms"`, or `"bonds"`.

XII.2 Anchor Helpers

`#atom-anchor((index), (anchor): "mid")` → `anchor`

Select an atom anchor for annotation placement.

Argument

`(index)` `int`

Atom index shown by `(show-indices)`.

Argument

`(anchor): "mid"` `str`

CeTZ anchor on the atom object, such as `"north"`, `"east"`, or `"mid"`.

`#bond-anchor((index), (anchor): "50%")` → `anchor`

Select a bond anchor for annotation placement.

Argument

`(index)` `int`

Bond index shown by `(show-indices)`.

Argument

`(anchor): "50%"` `str`

CeTZ anchor on the bond object.

`#molecule-anchor((anchor): "center")` → `anchor`

Select an anchor on the whole rendered molecule.

Argument

`(anchor): "center"` `str`

CeTZ group anchor such as `"center"`, `"north"`, `"east"`, `"south"`, or `"west"`.

`#atom-ref((index))` → `str`

Return the internal atom anchor name as a string.

`#bond-ref((index))` → `str`

Return the internal bond anchor name as a string.

XII.3 Annotation Builders

```
#callout-annotation(
  ⟨at⟩,
  ⟨label⟩,
  ⟨anchor⟩: "mid",
  ⟨side⟩: "north-east",
  ⟨label-at⟩: auto,
  ⟨leader⟩: "curve",
  ⟨mark⟩: none
) → annotation
```

Build an external label with a leader line.

Argument

⟨at⟩ — anchor

Target anchor.

Argument

⟨label⟩ — content

Label content.

Argument

⟨side⟩: "north-east" — str

Preset label side. Supported values are "east", "west", "north", "south", and diagonal combinations.

Argument

⟨label-at⟩: auto — auto | dictionary

Explicit CeTZ coordinate for the label.

Argument

⟨leader⟩: "curve" — str

Leader style: "curve", "straight", or "elbow".

Argument

⟨leader-end⟩: auto — auto | dictionary

Manual CeTZ coordinate where the leader should stop.

Argument

⟨target-gap⟩: 0.2 — float

Clearance from the target when ⟨leader-end⟩ is automatic.

```
#arrow-annotation(⟨from⟩, ⟨to⟩, ⟨label⟩: none, ⟨mark⟩: (end: ">>", fill:
luma(0%))) → annotation
```

Build a free arrow overlay.

#label-annotation(⟨at⟩, ⟨label⟩, ⟨offset⟩: (0, 0.45)) → annotation

Build a free text label overlay without a leader line.

#cetz-annotation(⟨body⟩) → annotation

Run custom CeTZ code after the molecule is drawn.

Argument

⟨body⟩ function

Function receiving the generated molecule name.

Part XIII

Limitations

- Full mode can become crowded for large molecules because explicit hydrogens and atom labels occupy real page space. Prefer abbreviated or skeletal mode, or increase `(atom-sep)`.
- A valid Molfile/SDF 2D layout is preserved even when its full-mode labels are crowded. Automatic relayout is limited to collapsed or numerically unstable XY coordinates.
- SMILES and unusable SDF coordinates are laid out with Coordgen. The result is deterministic for a given bundled plugin, but it may differ from an external chemical drawing program.
- Relayout preserves explicit SDF wedge, parity, and enhanced-stereo metadata. It does not infer stereochemistry solely from 3D coordinates.
- Extended-chirality layout rotates ligand branches around a stereocenter. Invalid or cyclic topology may therefore fall back to a textual chirality annotation when its branches cannot be moved independently.
- Annotation helpers cover common callouts and arrows, not automatic collision-free figure composition. For complex layouts, use `#cetz-annotation` or dump the generated `alchemist` code.
- Rendering CI catches compilation failures and package/CLI source divergence on the listed Typst versions. It is not a pixel-snapshot guarantee, so review fonts and final PDF appearance for publication output.
- Maintainers can run `scripts/check-pubchem-visual-regression.py` with the ignored local PubChem corpus for opt-in pixel regression checks; its baseline is machine-local and is not distributed with the package.

Part XIV

License and Dependencies

`molchemist` is distributed under the MIT license. Molfile / SDF parsing is powered by `sdfrust`; SMILES parsing is based on `opensmiles`; SMILES 2D coordinate generation uses `CoordgenLibs`; rendering is handled by `alchemist` and CeTZ. See the third-party notices distributed with the package for full license details.

The example SDF files and rendered example images are attributed separately from the package code. In particular, this manual includes PubChem-derived example structures such as CID 241[6]. See `THIRD_PARTY_NOTICES.md` and `docs/assets/README.md` for source URLs and the relevant NCBI data-usage policy.

[6] https://pubchem.ncbi.nlm.nih.gov/compound/241

Part XV

Index

A

B

C

L

M

R